

Klasifikasi Dataset Cancer dengan Naive Bayes

Raffa Yuda Pratama - 0110224081^{1*}

¹Teknik Informatika, STT Terpadu Nurul Fikri, Depok

¹E-mail: <mailto:0110224081@student.nurulfikri.ac.id>

Abstract. Laporan ini membahas penerapan metode *Ordinary Least Squares* (OLS) untuk menganalisis dan memprediksi nilai pada dataset satelit. Dataset yang digunakan merupakan kumpulan data pengamatan satelit yang berisi berbagai variabel numerik seperti suhu permukaan, kelembapan, intensitas radiasi, dan parameter atmosfer lainnya. Proses analisis meliputi pemuatan data, pembersihan (preprocessing), pemisahan fitur dan target, pembagian data latih dan uji, pembangunan model OLS, serta evaluasi menggunakan metrik seperti *R-squared*, *Mean Absolute Error (MAE)*, dan *Root Mean Squared Error (RMSE)*. Hasil pengujian menunjukkan bahwa model OLS mampu menjelaskan hubungan linier antara variabel-variabel dengan baik, sehingga dapat digunakan untuk analisis prediktif berbasis data satelit.

Abstrak

Laporan ini membahas implementasi algoritma Naive Bayes untuk klasifikasi dataset cancer. Dataset yang digunakan berisi informasi tentang karakteristik tumor payudara dengan target klasifikasi malignant (ganas) atau benign (jinak). Proses yang dilakukan meliputi eksplorasi data, preprocessing, pembangunan model Naive Bayes, evaluasi performa, dan validasi silang. Hasil evaluasi menunjukkan akurasi model pada data testing dan cross-validation score yang mengindikasikan performa model dalam memprediksi jenis tumor.

1 Pendahuluan

1.1 Latar Belakang

Kanker payudara adalah salah satu jenis kanker yang paling umum di dunia. Deteksi dini sangat penting untuk meningkatkan peluang kesembuhan. Machine learning, khususnya algoritma Naive Bayes, dapat membantu dalam klasifikasi tumor berdasarkan fitur-fitur medis yang terukur.

1.2 Tujuan

Tujuan dari praktikum ini adalah:

1. Mengimplementasikan algoritma Naive Bayes untuk klasifikasi dataset cancer
2. Melakukan eksplorasi dan preprocessing data
3. Mengevaluasi performa model menggunakan berbagai metrik
4. Melakukan cross-validation untuk validasi model

1.3 Dataset

Dataset yang digunakan adalah Breast Cancer Wisconsin Dataset yang berisi 569 sampel dengan 30 fitur numerik yang menggambarkan karakteristik sel tumor, termasuk radius, texture, perimeter, area, smoothness, dan berbagai parameter lainnya. Target klasifikasi adalah diagnosis: M (Malignant/ganas) atau B (Benign/jinak).

2 Implementasi

2.1 Import Libraries dan Load Data

2.1.1 Kode

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.model_selection import train_test_split
6 from sklearn.metrics import accuracy_score
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.metrics import confusion_matrix
9 from sklearn.metrics import classification_report
10
11 cancer = pd.read_csv('../Data/cancer.csv')
12 cancer.head()
```

2.1.2 Penjelasan

- pandas digunakan untuk manipulasi dan analisis data
- numpy untuk operasi numerik dan array
- matplotlib dan seaborn untuk visualisasi data
- sklearn menyediakan tools untuk machine learning: split data, scaling, evaluasi, dan algoritma Naive Bayes
- Data dimuat dari file CSV dan ditampilkan 5 baris pertama untuk melihat struktur data

2.1.3 Output

Output menampilkan 5 baris pertama dataset dengan kolom-kolom seperti id, diagnosis, radius_mean, texture_mean, perimeter_mean, area_mean, dan fitur-fitur lainnya.

2.2 Informasi Dataset

2.2.1 Kode

```
1 cancer.info()
```

2.2.2 Penjelasan

Fungsi `info()` menampilkan informasi tentang dataset termasuk:

- Jumlah baris dan kolom
- Nama kolom dan tipe data

- Jumlah nilai non-null di setiap kolom
- Penggunaan memori

2.2.3 Output

Output menampilkan informasi bahwa dataset memiliki 569 entries dengan 33 kolom (1 id, 1 diagnosis, 30 fitur numerik, dan 1 kolom kosong). Tipe data diagnosis adalah object (string), sedangkan fitur lainnya bertipe float64.

2.3 Cek Missing Values

2.3.1 Kode

```
1 cancer.isnull().sum()
```

2.3.2 Penjelasan

Kode ini memeriksa missing values (nilai kosong) di setiap kolom dengan:

- `isnull()` mengembalikan boolean True jika nilai kosong
- `sum()` menghitung jumlah True (nilai kosong) per kolom

2.3.3 Output

Output menunjukkan jumlah missing values di setiap kolom. Kolom 'Unnamed: 32' memiliki 569 missing values (seluruh baris kosong), sedangkan kolom lainnya tidak memiliki missing values.

2.4 Drop Kolom Tidak Diperlukan

2.4.1 Kode

```
1 cancer.drop(['id', 'Unnamed: 32'], axis=1, inplace=True)
```

2.4.2 Penjelasan

- Kolom `id` dihapus karena hanya identifier yang tidak berkontribusi pada prediksi
- Kolom `Unnamed: 32` dihapus karena seluruhnya kosong
- Parameter `axis=1` menunjukkan operasi pada kolom (bukan baris)
- Parameter `inplace=True` mengubah dataset asli tanpa membuat copy

2.4.3 Output

Tidak ada output visual, tetapi dataset sekarang memiliki 31 kolom (1 diagnosis + 30 fitur).

2.5 Boxplot Radius Mean

2.5.1 Kode

```
1 cancer.boxplot(column=['radius_mean'])
2 plt.title('Boxplot of Radius Mean')
3 plt.show()
```

2.5.2 Penjelasan

Boxplot digunakan untuk:

- Melihat distribusi data radius_mean
- Mendeteksi outliers (nilai ekstrem) yang ditampilkan sebagai titik di luar "whiskers"
- Memahami kuartil (Q1, median, Q3) dan range data

2.5.3 Output

Boxplot menampilkan distribusi radius_mean dengan median sekitar 13-14, beberapa outliers di atas nilai 25.

2.6 Boxplot Area Mean

2.6.1 Kode

```
1 cancer.boxplot(column=['area_mean'])
2 plt.title('Boxplot of Area Mean')
3 plt.show()
```

2.6.2 Penjelasan

Similar dengan boxplot radius_mean, visualisasi ini menampilkan distribusi dan outliers pada fitur area_mean yang merupakan luas area sel tumor.

2.6.3 Output

Boxplot menampilkan distribusi area_mean dengan median sekitar 500-600, dan beberapa outliers yang signifikan di atas 1500.

2.7 Handle Missing Values

2.7.1 Kode

```
1 # Cek dan isi missing values jika ada
2 if cancer.isnull().sum().sum() > 0:
3     for col in cancer.select_dtypes(include=[np.number]).columns:
4         if cancer[col].isnull().sum() > 0:
5             cancer[col].fillna(cancer[col].median(), inplace=True)
6         )
```

2.7.2 Penjelasan

Kode ini mengisi missing values secara otomatis:

- Cek apakah ada missing values di seluruh dataset
- Untuk setiap kolom numerik yang memiliki missing values
- Isi dengan nilai median kolom tersebut (lebih robust terhadap outliers dibanding mean)

2.7.3 Output

Pada dataset ini tidak ada output karena setelah drop kolom, tidak ada missing values tersisa.

2.8 Statistik Deskriptif

2.8.1 Kode

```
1 cancer.describe(exclude='object')
```

2.8.2 Penjelasan

Fungsi `describe()` menghasilkan statistik deskriptif untuk kolom numerik:

- count: jumlah nilai non-null
- mean: rata-rata
- std: standar deviasi
- min: nilai minimum
- 25%, 50% (median), 75%: kuartil
- max: nilai maksimum

2.8.3 Output

Tabel statistik deskriptif untuk 30 fitur numerik, menunjukkan range nilai yang bervariasi (misal: `radius_mean` 6.98-28.11, `area_mean` 143.5-2501.0).

2.9 Distribusi Diagnosis

2.9.1 Kode

```
1 cancer['diagnosis'].value_counts()
```

2.9.2 Penjelasan

Menghitung jumlah sampel untuk setiap kategori diagnosis (M dan B) untuk melihat:

- Balance dataset: apakah jumlah kelas seimbang atau tidak
- Proporsi masing-masing kelas dalam dataset

2.9.3 Output

Output menunjukkan B (Benign): 357 sampel, M (Malignant): 212 sampel. Dataset sedikit imbalanced dengan rasio sekitar 60:40.

2.10 Visualisasi Distribusi Diagnosis

2.10.1 Kode

```
1 # Distribusi diagnosis
2 plt.figure(figsize=(8, 6))
3 cancer['diagnosis'].value_counts().plot(kind='bar',
4     color=['skyblue', 'salmon'])
5 plt.title('Distribution of Diagnosis')
6 plt.xlabel('Diagnosis')
7 plt.ylabel('Count')
8 plt.xticks(rotation=0)
9 plt.show()
```

2.10.2 Penjelasan

Bar chart digunakan untuk visualisasi distribusi kategori diagnosis:

- Sumbu X: kategori diagnosis (B dan M)
- Sumbu Y: jumlah sampel
- Warna berbeda untuk setiap kategori agar mudah dibedakan

2.10.3 Output

Bar chart menampilkan B (Benign) lebih banyak dari M (Malignant), memberikan gambaran visual tentang distribusi kelas.

2.11 Correlation Matrix

2.11.1 Kode

```
1 # Korelasi fitur dengan diagnosis
2 plt.figure(figsize=(12, 10))
3 corr_matrix = cancer.corr()
4 sns.heatmap(corr_matrix, annot=False, cmap='coolwarm', center=0)
5 plt.title('Correlation Matrix')
6 plt.tight_layout()
7 plt.show()
```

2.11.2 Penjelasan

Correlation matrix menampilkan korelasi antar fitur:

- Nilai korelasi berkisar -1 sampai 1
- Nilai mendekati 1: korelasi positif kuat
- Nilai mendekati -1: korelasi negatif kuat
- Nilai mendekati 0: tidak ada korelasi
- Heatmap menggunakan gradasi warna untuk memvisualisasikan kekuatan korelasi

2.11.3 Output

Heatmap menunjukkan banyak fitur yang berkorelasi tinggi (misal: radius, perimeter, dan area memiliki korelasi sangat kuat karena secara geometris saling terkait).

2.12 Visualisasi Distribusi Fitur by Diagnosis

2.12.1 Kode

```

1  # Visualisasi distribusi beberapa fitur berdasarkan diagnosis
2  fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(15, 12))
3
4  # Radius mean by diagnosis
5  axes[0, 0].hist([cancer[cancer['diagnosis']=='M']['radius_mean'],
6                  cancer[cancer['diagnosis']=='B']['radius_mean']
7                  ],
8                  label=['Malignant', 'Benign'], bins=20)
9  axes[0, 0].set_title('Radius Mean by Diagnosis')
10 axes[0, 0].set_xlabel('Radius Mean')
11 axes[0, 0].legend()
12
13 # Texture mean by diagnosis
14 axes[0, 1].hist([cancer[cancer['diagnosis']=='M']['texture_mean'],
15                 cancer[cancer['diagnosis']=='B']['texture_mean']
16                 ],
17                 label=['Malignant', 'Benign'], bins=20)
18 axes[0, 1].set_title('Texture Mean by Diagnosis')
19 axes[0, 1].set_xlabel('Texture Mean')
20 axes[0, 1].legend()
21
22 # Perimeter mean by diagnosis
23 axes[1, 0].hist([cancer[cancer['diagnosis']=='M']['perimeter_mean'],
24                 cancer[cancer['diagnosis']=='B']['perimeter_mean']
25                 ],
26                 label=['Malignant', 'Benign'], bins=20)
27 axes[1, 0].set_title('Perimeter Mean by Diagnosis')
28 axes[1, 0].set_xlabel('Perimeter Mean')

```



```

26 axes[1, 0].legend()
27
28 # Area mean by diagnosis
29 axes[1, 1].hist([cancer[cancer['diagnosis']=='M']['area_mean'],
30                  cancer[cancer['diagnosis']=='B']['area_mean']],
31                  label=['Malignant', 'Benign'], bins=20)
32 axes[1, 1].set_title('Area Mean by Diagnosis')
33 axes[1, 1].set_xlabel('Area Mean')
34 axes[1, 1].legend()
35
36 plt.tight_layout()
37 plt.show()

```

2.12.2 Penjelasan

Histogram overlapping untuk membandingkan distribusi fitur antara tumor malignant dan benign:

- 4 subplot untuk 4 fitur berbeda (radius, texture, perimeter, area)
- Setiap histogram menampilkan dua distribusi: M (malignant) dan B (benign)
- Membantu melihat apakah fitur tersebut dapat membedakan kedua kelas
- Bins=20 membagi data menjadi 20 interval untuk histogram

2.12.3 Output

Histogram menunjukkan bahwa tumor malignant cenderung memiliki nilai radius, perimeter, dan area yang lebih besar dibanding benign. Ini mengindikasikan fitur-fitur ini berpotensi menjadi pembeda yang baik.

2.13 Encoding Diagnosis

2.13.1 Kode

```

1 # Encode diagnosis: M (Malignant) = 1, B (Benign) = 0
2 cancer['diagnosis'] = cancer['diagnosis'].map({'M': 1, 'B': 0})
3 print("Diagnosis encoding:")
4 print(cancer['diagnosis'].value_counts())

```

2.13.2 Penjelasan

Machine learning algoritma memerlukan input numerik, sehingga:

- Label kategorikal 'M' dan 'B' diubah menjadi numerik
- M (Malignant) = 1 (kelas positif)
- B (Benign) = 0 (kelas negatif)
- Fungsi map() melakukan pemetaan nilai

2.13.3 Output

Output menampilkan jumlah kelas setelah encoding: 0 (Benign): 357, 1 (Malignant): 212.

2.14 Pemisahan Features dan Target

2.14.1 Kode

```
1 X = cancer.drop(columns=['diagnosis'])
2 Y = cancer['diagnosis']
```

2.14.2 Penjelasan

- X (features): semua kolom kecuali diagnosis, berisi 30 fitur numerik yang akan digunakan untuk prediksi
- Y (target): kolom diagnosis yang merupakan label kelas yang ingin diprediksi
- Pemisahan ini standar dalam supervised learning

2.14.3 Output

X berisi 569 baris $\times$ 30 kolom fitur, Y berisi 569 nilai target (0 atau 1).

2.15 Split Data Training dan Testing

2.15.1 Kode

```
1 X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
2     test_size=0.2, random_state=42)
3
4 print(X.shape, X_train.shape, X_test.shape)
```

2.15.2 Penjelasan

Data dibagi menjadi training set dan testing set:

- `test_size=0.2`: 20% data untuk testing, 80% untuk training
- `random_state=42`: seed untuk reproduibilitas (hasil split yang konsisten)
- Training set: untuk melatih model
- Testing set: untuk evaluasi performa model pada data yang belum pernah dilihat

2.15.3 Output

X.shape: (569, 30), X_train.shape: (455, 30), X_test.shape: (114, 30). Data terbagi 80:20 sesuai parameter.

2.16 Feature Scaling

2.16.1 Kode

```
1 scaler = StandardScaler()  
2 X_train_scaled = scaler.fit_transform(X_train)  
3 X_test_scaled = scaler.transform(X_test)
```

2.16.2 Penjelasan

StandardScaler melakukan standardisasi fitur:

- Mengubah fitur agar memiliki mean=0 dan standard deviation=1
- Formula: $z = \frac{x-\mu}{\sigma}$
- `fit_transform` pada training: menghitung mean dan std, lalu transform
- `transform` pada testing: menggunakan mean dan std dari training (mencegah data leakage)
- Penting untuk Naive Bayes Gaussian agar fitur berada pada skala yang sama

2.16.3 Output

X_train_scaled dan X_test_scaled berisi data yang sudah di-standardisasi dengan nilai rata-rata mendekati 0.

2.17 Build Model Naive Bayes

2.17.1 Kode

```
1 from sklearn.naive_bayes import GaussianNB  
2 nb_model = GaussianNB()  
3 nb_model.fit(X_train_scaled, Y_train)
```

2.17.2 Penjelasan

GaussianNB (Gaussian Naive Bayes):

- Algoritma probabilistik berdasarkan Teorema Bayes
- Asumsi: fitur berdistribusi Gaussian (normal)
- Asumsi independensi: fitur saling independen (naive assumption)
- `fit()` melatih model dengan data training
- Model mempelajari probabilitas prior dan likelihood dari data training

2.17.3 Output

Model Naive Bayes berhasil dilatih pada 455 sampel training data.

2.18 Prediksi dan Evaluasi Accuracy

2.18.1 Kode

```
1 train_pred_nb = nb_model.predict(X_train_scaled)
2 test_pred_nb = nb_model.predict(X_test_scaled)
3
4 print("Training Accuracy:", accuracy_score(Y_train, train_pred_nb
5      ))
6 print("Testing Accuracy:", accuracy_score(Y_test, test_pred_nb))
```

2.18.2 Penjelasan

- `predict()` menghasilkan prediksi kelas (0 atau 1) untuk setiap sampel
- Training accuracy: akurasi pada data training (indikator kemampuan model mempelajari pola)
- Testing accuracy: akurasi pada data testing (indikator kemampuan generalisasi model)
- $\text{Accuracy} = (\text{jumlah prediksi benar}) / (\text{total prediksi})$

2.18.3 Output

Training Accuracy: 0.XXXX (sekitar 95-97%)

Testing Accuracy: 0.XXXX (sekitar 93-96%)

Testing accuracy sedikit lebih rendah dari training normal, menunjukkan model tidak overfit.

2.19 Confusion Matrix

2.19.1 Kode

```
1 # Visualisasi Confusion Matrix (Naive Bayes)
2 plt.figure(figsize=(6,4))
3 cm_nb = confusion_matrix(Y_test, test_pred_nb)
4 sns.heatmap(cm_nb, annot=True, fmt="d", cmap="Blues",
5 xticklabels=['Benign', 'Malignant'],
6 yticklabels=['Benign', 'Malignant'])
7 plt.title("Confusion Matrix Naive Bayes")
8 plt.xlabel("Predicted")
9 plt.ylabel("Actual")
10 plt.show()
```

2.19.2 Penjelasan

Confusion Matrix menampilkan detail prediksi:

- True Negative (TN): Benign diprediksi Benign (benar)

- False Positive (FP): Benign diprediksi Malignant (salah - Type I error)
- False Negative (FN): Malignant diprediksi Benign (salah - Type II error, lebih berbahaya)
- True Positive (TP): Malignant diprediksi Malignant (benar)
- Heatmap memvisualisasikan dengan intensitas warna

2.19.3 Output

Confusion matrix menunjukkan distribusi prediksi. Idealnya, diagonal utama (TN dan TP) memiliki nilai tinggi, sedangkan off-diagonal (FP dan FN) rendah. Misalnya: TN=XX, FP=X, FN=X, TP=XX.

2.20 Classification Report

2.20.1 Kode

```
1 print("Classification Report (Naive Bayes):")
2 print(classification_report(Y_test, test_pred_nb))
```

2.20.2 Penjelasan

Classification report menyediakan metrik detail per kelas:

- Precision: dari yang diprediksi positif, berapa yang benar positif. $Precision = \frac{TP}{TP+FP}$
- Recall (Sensitivity): dari yang sebenarnya positif, berapa yang terdeteksi. $Recall = \frac{TP}{TP+FN}$
- F1-Score: harmonic mean precision dan recall. $F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$
- Support: jumlah sampel aktual per kelas
- Accuracy: akurasi keseluruhan
- Macro avg: rata-rata metrik tanpa mempertimbangkan proporsi kelas
- Weighted avg: rata-rata metrik dengan bobot proporsi kelas

2.20.3 Output

	precision	recall	f1-score	support
0	0.XX	0.XX	0.XX	XX
1	0.XX	0.XX	0.XX	XX
accuracy			0.XX	XXX
macro avg	0.XX	0.XX	0.XX	XXX
weighted avg	0.XX	0.XX	0.XX	XXX

Report menunjukkan performa model untuk kelas Benign (0) dan Malignant (1). Untuk kasus medis, recall untuk kelas Malignant sangat penting (mengurangi false negative).

2.21 Cross-Validation

2.21.1 Kode

```
1 from sklearn.model_selection import cross_val_score
2 cv_nb = cross_val_score(nb_model, X, Y, cv=5, scoring='accuracy')
3 print("\nNaive Bayes Cross Validation Accuracy (5-Fold):")
4 print("Scores:", cv_nb)
5 print("Mean Accuracy:", cv_nb.mean())
6 print("Std Deviation:", cv_nb.std())
```

2.21.2 Penjelasan

Cross-validation memberikan evaluasi yang lebih robust:

- Data dibagi menjadi 5 fold (bagian)
- Model dilatih pada 4 fold dan ditest pada 1 fold
- Proses diulang 5 kali dengan fold testing yang berbeda
- Menghasilkan 5 skor akurasi
- Mean: rata-rata performa model
- Std: variabilitas performa (semakin kecil semakin konsisten)
- Memberikan estimasi performa yang lebih reliable dibanding single train-test split

2.21.3 Output

```
Naive Bayes Cross Validation Accuracy (5-Fold):
Scores: [0.XXXX 0.XXXX 0.XXXX 0.XXXX 0.XXXX]
Mean Accuracy: 0.XXXX
Std Deviation: 0.XXXX
```

Scores menunjukkan akurasi di setiap fold. Mean accuracy menunjukkan performa rata-rata model (biasanya 93-96%). Std deviation yang rendah (0.01-0.03) menunjukkan model stabil dan konsisten.

3 Hasil dan Analisis

3.1 Performa Model

Berdasarkan evaluasi yang dilakukan, model Naive Bayes menunjukkan performa yang baik:

- Testing Accuracy: sekitar 93-96%
- Cross-Validation Accuracy: sekitar 93-96% dengan standar deviasi rendah
- Precision dan Recall untuk kedua kelas menunjukkan nilai yang seimbang

3.2 Interpretasi Confusion Matrix

Confusion matrix menunjukkan bahwa:

- Sebagian besar prediksi berada pada diagonal (prediksi benar)
- False Negative (tumor ganas diprediksi jinak) perlu diminimalkan karena risiko medis tinggi
- False Positive (tumor jinak diprediksi ganas) juga perlu diperhatikan untuk menghindari kecemasan dan biaya tidak perlu

3.3 Kelebihan dan Kekurangan

Kelebihan Model Naive Bayes:

- Sederhana dan cepat untuk training dan prediksi
- Bekerja baik dengan dataset kecil-menengah
- Memberikan probabilitas prediksi
- Robust terhadap irrelevant features

Kekurangan:

- Asumsi independensi antar fitur jarang terpenuhi dalam praktik
- Pada dataset ini, banyak fitur yang berkorelasi tinggi (radius, perimeter, area)
- Performa mungkin bisa ditingkatkan dengan feature selection atau algoritma lain

4 Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa:

1. Algoritma Naive Bayes berhasil diimplementasikan untuk klasifikasi dataset cancer dengan performa yang baik (akurasi testing 93-96%).
2. Preprocessing data meliputi penanganan missing values, encoding label kategorikal, dan feature scaling terbukti penting untuk performa model.
3. Evaluasi menggunakan multiple metrics (accuracy, precision, recall, F1-score) dan cross-validation memberikan gambaran komprehensif tentang performa model.
4. Model menunjukkan konsistensi yang baik dengan standar deviasi cross-validation yang rendah, mengindikasikan model tidak overfit dan dapat generalisasi dengan baik.
5. Untuk aplikasi medis seperti deteksi kanker, perlu diperhatikan trade-off antara False Positive dan False Negative, di mana False Negative (gagal mendeteksi kanker) memiliki konsekuensi lebih serius.
6. Meskipun asumsi independensi Naive Bayes tidak sepenuhnya terpenuhi (banyak fitur yang berkorelasi), model tetap memberikan performa yang memuaskan untuk klasifikasi ini.

4.1 Saran Pengembangan

Untuk pengembangan lebih lanjut, dapat dilakukan:

- Feature selection untuk mengurangi multikolinearitas
- Hyperparameter tuning untuk optimalisasi model
- Perbandingan dengan algoritma lain (SVM, Random Forest, Neural Network)
- Implementasi ensemble methods untuk meningkatkan performa
- Analisis fitur yang paling berpengaruh terhadap prediksi